

Module 1

Automation with Selenium

Module 1 of 4 — Selenium / QA Automation Training
Detailed Curriculum Report with Explanations, Examples & Code

1.1 Introduction & Different Flavors of Selenium

Selenium is a free, open-source suite of tools for automating web browsers. It is the most widely used framework for functional and regression testing of web applications because it supports multiple programming languages (Python, Java, C#, JavaScript), multiple browsers (Chrome, Firefox, Edge, Safari), and multiple operating systems.

Why Selenium?

- Open source and free — no licensing cost, large community support.
- Cross-browser and cross-platform — the same script can run on Chrome, Firefox, Edge, on Windows, Linux or macOS.
- Multi-language support — bindings available for Python, Java, C#, JavaScript, Ruby, Kotlin.
- Integrates with test frameworks (PyTest, JUnit, TestNG), CI/CD tools (Jenkins, GitHub Actions) and reporting tools (Allure, HTML reports).
- Supports parallel and distributed execution through Selenium Grid.

The Selenium Suite — Components

Component	Purpose
Selenium IDE	Record-and-playback browser extension (Chrome/Firefox) for quickly building and debugging simple tests
Selenium RC (Remote Control)	The original (legacy, now deprecated) tool that used a server to inject JavaScript into the browser to drive tests
Selenium WebDriver	The core API that talks directly to the browser via native browser drivers (ChromeDriver, GeckoDriver, etc.)
Selenium Grid	Infrastructure to run tests in parallel across multiple machines/browsers/OS combinations (a Hub-Node architecture)

Note: Selenium RC is deprecated — new projects should always use Selenium WebDriver (currently Selenium 4).

1.2 Selenium WebDriver — Introduction & Installation

Selenium 4 WebDriver communicates directly with the browser using the standardized W3C WebDriver protocol (earlier versions used the JSON Wire Protocol, which added an extra translation layer). This makes Selenium 4 faster and more stable, and it removes the need to manage driver binaries manually thanks to Selenium Manager (built in since Selenium 4.6).

WebDriver Architecture

- **Client (your script)** — the Python code using the `selenium` library sends commands.
- **JSON / HTTP over W3C protocol** — commands are serialized as HTTP requests.
- **Browser Driver** (ChromeDriver / GeckoDriver / EdgeDriver) — a small server that receives the commands and translates them into native browser calls.
- **Browser** — actually executes the action (click, type, navigate) and returns the result back up the chain.

Software Requirements

- Python 3.8+ installed and added to PATH.
- pip (Python package manager).
- An IDE — PyCharm or VS Code.
- Google Chrome / Firefox / Edge browser installed.
- selenium package (v4.x).

Installation

```
# Installing Selenium WebDriver with pip
pip install selenium

# Verify installation
python -c "import selenium; print(selenium.__version__)"
```

Setting Drivers in the System Path (Windows)

From Selenium 4.6 onward, **Selenium Manager** auto-detects your installed browser version and downloads the matching driver binary for you, so manual PATH configuration is usually unnecessary. For older setups or explicit control, you can still download the driver executable and place its folder in the Windows PATH environment variable, or pass its path directly via a Service object.

driver_setup.py

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service

# Explicit driver path (only needed if Selenium Manager can't auto-resolve one)
service = Service(executable_path=r"C:\drivers\chromedriver.exe")
driver = webdriver.Chrome(service=service)

# Modern (Selenium 4.6+) - Selenium Manager resolves the driver automatically
driver = webdriver.Chrome()
```

Running Tests in Different Browsers (Windows)

cross_browser.py

```
from selenium import webdriver

# Chrome
driver = webdriver.Chrome()

# Firefox
driver = webdriver.Firefox()

# Microsoft Edge
driver = webdriver.Edge()

driver.get("https://www.saucedemo.com")
print(driver.title)
driver.quit()
```

1.3 Locators and Object Identification

A **locator** is the strategy Selenium uses to find a specific HTML element on a page so it can interact with it. Choosing a stable, unique locator is the single most important skill in Selenium automation — fragile locators are the number one cause of flaky tests.

Challenges in Locating Complex Web Elements

- Elements with no unique id or name.
- Dynamically generated IDs/classes that change on every page load (common in React/Angular apps).
- Elements nested deep inside iframes or Shadow DOM.
- Multiple elements matching the same locator (needs indexing or a more specific relative path).

By ID, Name, Tag Name, Link Text, Class Name

basic_locators.py

```
from selenium.webdriver.common.by import By

driver.find_element(By.ID, "user-name")
driver.find_element(By.NAME, "password")
driver.find_element(By.TAG_NAME, "input")
driver.find_element(By.LINK_TEXT, "Sign in")
driver.find_element(By.PARTIAL_LINK_TEXT, "Sign")
driver.find_element(By.CLASS_NAME, "btn_action")
```

Finding a List of Elements

find_elements.py

```
items = driver.find_elements(By.CLASS_NAME, "inventory_item")
print(f"Total products: {len(items)}")
for item in items:
    print(item.text)
```

Note: `find_element` (singular) raises `NoSuchElementException` if nothing matches; `find_elements` (plural) returns an empty list instead — useful for existence checks without `try/except`.

CSS Selectors, Wildcards & Child Nodes

css_selectors.py

```
# By CSS
driver.find_element(By.CSS_SELECTOR, "#user-name")
driver.find_element(By.CSS_SELECTOR, ".btn_action")
driver.find_element(By.CSS_SELECTOR, "input[data-test='username']")

# Wildcards with CSS (attribute contains/starts-with/ends-with)
driver.find_element(By.CSS_SELECTOR, "input[id*='user']") # contains
driver.find_element(By.CSS_SELECTOR, "input[id^='user']") # starts with
driver.find_element(By.CSS_SELECTOR, "input[id$='name']") # ends with

# Child nodes via CSS (direct child '>' vs any descendant ' ')
driver.find_element(By.CSS_SELECTOR, "div.login_wrapper > input")
driver.find_element(By.CSS_SELECTOR, "div.login_wrapper input")
```

1.4 Handling Different Controls on a Web Page

XPath — Absolute vs Relative

- **Absolute XPath** starts from the document root (/html/body/div/...) — brittle, breaks with any DOM change.
- **Relative XPath** starts with // and searches anywhere in the DOM — the recommended approach.

xpath_examples.py

```
# Absolute (avoid)
driver.find_element(By.XPATH, "/html/body/div[2]/div/form/div[1]/input")

# Relative (preferred)
driver.find_element(By.XPATH, "//input[@id='user-name']")

# contains()
driver.find_element(By.XPATH, "//input[contains(@id,'user')]")

# starts-with()
driver.find_element(By.XPATH, "//input[starts-with(@id,'user')]")

# Parent / sibling axes
driver.find_element(By.XPATH, "//label[text()='Username']/parent::div")
driver.find_element(By.XPATH, "//label[text()='Username']/following-sibling::input")
```

Handling Buttons, Input Boxes, Checkboxes, Radio Buttons, Select Boxes

controls.py

```
from selenium.webdriver.support.ui import Select

# Button
driver.find_element(By.ID, "login-button").click()

# Input box
box = driver.find_element(By.ID, "user-name")
box.clear()
box.send_keys("standard_user")

# Checkbox
cb = driver.find_element(By.ID, "subscribe")
if not cb.is_selected():
    cb.click()
print(cb.is_selected())          # verify state -> True

# Radio button
driver.find_element(By.CSS_SELECTOR, "input[value='male']").click()

# Select / dropdown box
dropdown = Select(driver.find_element(By.ID, "country"))
dropdown.select_by_visible_text("India")
dropdown.select_by_value("IN")
dropdown.select_by_index(2)
print(dropdown.first_selected_option.text)
```

1.5 Handling Different Controls — Advanced

Alert Boxes (JavaScript Alerts / Confirms / Prompts)

alerts.py

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver.find_element(By.ID, "alertBtn").click()
alert = WebDriverWait(driver, 10).until(EC.alert_is_present())
print(alert.text)
alert.accept()          # click OK

driver.find_element(By.ID, "confirmBtn").click()
WebDriverWait(driver, 10).until(EC.alert_is_present()).dismiss() # click Cancel

driver.find_element(By.ID, "promptBtn").click()
prompt = WebDriverWait(driver, 10).until(EC.alert_is_present())
prompt.send_keys("Pratik")
prompt.accept()
```

Date Pickers

datepicker.py

```
# Common approach: click to open, then click the target date cell
driver.find_element(By.ID, "datepicker").click()
driver.find_element(By.XPATH, "//a[text()='15']").click()

# When the widget allows direct typing:
date_field = driver.find_element(By.ID, "datepicker")
date_field.clear()
date_field.send_keys("09/24/2026")
```

File Upload

file_upload.py

```
import os
upload_input = driver.find_element(By.ID, "file-upload")
upload_input.send_keys(os.path.abspath("sample.pdf")) # bypasses the native OS dialog
```

Multiple Windows / Tabs

windows.py

```

main_window = driver.current_window_handle
driver.find_element(By.LINK_TEXT, "Open new tab").click()

for handle in driver.window_handles:
    if handle != main_window:
        driver.switch_to.window(handle)
        break

print(driver.title)
driver.close() # close the new tab
driver.switch_to.window(main_window) # go back to the original

```

Keyboard Events

keyboard.py

```

from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.action_chains import ActionChains

box = driver.find_element(By.ID, "search")
box.send_keys("selenium" + Keys.ENTER)

ActionChains(driver).key_down(Keys.CONTROL).send_keys('a').key_up(Keys.CONTROL).perform()

```

Scrollbar

scroll.py

```

# Scroll to a specific element
elem = driver.find_element(By.ID, "footer")
driver.execute_script("arguments[0].scrollIntoView(true);", elem)

# Scroll by pixel offset
driver.execute_script("window.scrollTo(0, 800);")

```

Drag and Drop

drag_drop.py

```

from selenium.webdriver.support.ui import WebDriverWait
source = driver.find_element(By.ID, "draggable")
target = driver.find_element(By.ID, "droppable")

ActionChains(driver).drag_and_drop(source, target).perform()

```

iFrames

iframes.py

```
driver.switch_to.frame("frameName")      # by name/id
# driver.switch_to.frame(0)              # by index
# driver.switch_to.frame(driver.find_element(By.TAG_NAME, "iframe")) # by WebElement

driver.find_element(By.ID, "insideFrameBtn").click()
driver.switch_to.default_content()        # back to the main page
```

Element State & Attributes

element_state.py

```
elem = driver.find_element(By.ID, "submit")
print(elem.is_enabled())      # True/False
print(elem.is_displayed())
print(elem.text)              # visible text
print(elem.get_attribute("value")) # any HTML attribute
print(elem.get_attribute("outerHTML"))
```

1.6 Handling Dynamic Elements

Working with a Web Table

Web tables are typically built from nested `<table>/<tr>/<td>` elements. To read or search table data reliably, iterate rows and columns using relative XPath indices.

web_table.py

```
rows = driver.find_elements(By.XPATH, "//table[@id='product-table']/tbody/tr")
print(f"Row count: {len(rows)}")

for row in rows:
    cols = row.find_elements(By.TAG_NAME, "td")
    print([c.text for c in cols])

# Find the "Price" for a specific product name by traversing the row
target_row = driver.find_element(
    By.XPATH, "//table[@id='product-table']/tr[td[text()='Sauce Labs Backpack']]"
)
price = target_row.find_element(By.XPATH, "./td[3]").text
print(price)
```

1.7 Synchronization / Wait Types

Modern web applications load content asynchronously (AJAX). If Selenium tries to interact with an element before it exists or becomes visible, it throws `NoSuchElementException` or `ElementNotInteractableException`. Synchronization (waiting) solves this.

Implicit Wait

A single global setting: tells the WebDriver to poll the DOM for a given amount of time when searching for any element before throwing an exception. Applies for the lifetime of the driver session.

implicit_wait.py

```
driver.implicitly_wait(10) # seconds; applies to every find_element call
```

Explicit Wait

Waits for a specific condition to become true for a specific element before proceeding — far more reliable than `time.sleep()` because it proceeds the instant the condition is met instead of always waiting the full duration.

explicit_wait.py

```

from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

wait = WebDriverWait(driver, 15)

elem = wait.until(EC.visibility_of_element_located((By.ID, "result")))
wait.until(EC.element_to_be_clickable((By.ID, "submit"))).click()
wait.until(EC.text_to_be_present_in_element((By.ID, "status"), "Completed"))

```

Note: Never mix implicit and explicit waits in the same script — it causes unpredictable, inconsistent wait times. Prefer explicit waits with `expected_conditions`.

1.8 Screenshots & Exception Handling

Taking Screenshots

screenshots.py

```

driver.save_screenshot("homepage.png")

# Generic reusable method, e.g. to capture on every test failure
import time
def take_screenshot(driver, name):
    filename = f"{name}_{int(time.time())}.png"
    driver.save_screenshot(filename)
    return filename

```

Exception Handling — try / except / finally

exceptions.py

```

from selenium.common.exceptions import NoSuchElementException, TimeoutException

try:
    elem = driver.find_element(By.ID, "does-not-exist")
    elem.click()
except NoSuchElementException:
    print("Element was not found on the page.")
except TimeoutException:
    print("Timed out waiting for the element.")
finally:
    driver.quit() # always runs - guarantees browser cleanup

```

1.9 Advanced Interactions

Reading Data from Excel, CSV, JSON, XML, Properties Files

read_data.py

```
import json, csv
import openpyxl

# Excel
wb = openpyxl.load_workbook("testdata.xlsx")
sheet = wb.active
for row in sheet.iter_rows(min_row=2, values_only=True):
    username, password = row
    print(username, password)

# JSON
with open("testdata.json") as f:
    data = json.load(f)
print(data["users"][0]["username"])

# CSV
with open("testdata.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["username"], row["password"])
```

Writing Data into Excel / JSON

write_data.py

```
import openpyxl, json

wb = openpyxl.Workbook()
ws = wb.active
ws.append(["TestCase", "Result"])
ws.append(["Login_Valid", "PASS"])
wb.save("results.xlsx")

results = {"Login_Valid": "PASS", "Login_Invalid": "FAIL"}
with open("results.json", "w") as f:
    json.dump(results, f, indent=2)
```

Mouse Hover Actions

mouse_hover.py

```
from selenium.webdriver.common.action_chains import ActionChains

menu = driver.find_element(By.ID, "menu")
ActionChains(driver).move_to_element(menu).perform()
driver.find_element(By.LINK_TEXT, "Submenu Item").click()
```

Executing JavaScript Commands

javascript.py

```
driver.execute_script("document.getElementById('user-name').value='standard_user';")
driver.execute_script("arguments[0].click();", driver.find_element(By.ID, "login-button"))
value = driver.execute_script("return document.title;")
print(value)
```

Module 1 Summary: You now know the Selenium suite, WebDriver architecture, all major locator strategies (ID, Name, CSS, XPath), how to interact with every standard web control (text boxes, checkboxes, dropdowns, alerts, iframes, tables, calendars, uploads), synchronization strategies, screenshots, exception handling, and data-driven building blocks (Excel/JSON/CSV). This is the foundation for Module 2, where these scripts are organized into proper test frameworks.